

CS 166 Project Report

Group Information

Group 38

Amy Lau (alau030)

Sarah Pun (spun003)

Implementation Description

For this project phase three, we implemented the functions: createUser, login, viewProfile, updateProfile, viewCatalog, placeOrder, viewAllOrders, viewRecentOrders, viewOrderInfo, viewTrackingInfo, updateTrackingInfo, updateCatalog, and updateUser to rent and view the options at a game rental store. Functions were created to be non-SQL user friendly and robust to unexpected/invalid user input. We created trigger functions and indices to be able to execute the tables and improve the performance of retrieving the data from the tables. Overall, our implementation of the game rental management application allows the code to run efficiently and reduce space complexity.

Query Screenshots and Explanations

Insert into Users Table

```
// insert into Users table
String query = "INSERT INTO Users (login, password, role, favGames, phoneNum, numOverDueGames) " +
    "VALUES ('" + login + "', '" + password + "', 'customer', '', '" + phoneNum + "', 0);";
esql.executeUpdate(query);
System.out.println("User has been created!");
```

This query inserts user info into the user table using information that the user provides when they are first creating a new user, used in the CreateUser() function.

SELECT * FROM Users

```
// check if user login and password exists in Users table
String query = String.format("SELECT * FROM Users WHERE login = '%s' AND password = '%s';", login, password);
int userNum = esql.executeQuery(query);
if (userNum > 0) {
    System.out.println("You have successfully logged in!");
    return login;
}
```

This query is used for selecting user information that matches the login and password from the input. It is used in the function Login() to ensure that the user exists/is able to login.

Update

```
case 1:
    System.out.print("Enter new favorite games: ");
    String favGames = in.readLine();
    updateQuery = String.format("UPDATE Users SET favGames = '%s' WHERE login = '%s';", favGames, login);
    break;
case 2:
    System.out.print("Enter new password: ");
    String password = in.readLine();
    updateQuery = String.format("UPDATE Users SET password = '%s' WHERE login = '%s';", password, login);
    break;
case 3:
    System.out.print("Enter new phone number: ");
    String phoneNum = in.readLine();
    updateQuery = String.format("UPDATE Users SET phoneNum = '%s' WHERE login = '%s';", phoneNum, login);
    break;
```

These queries all update an attribute in the Users table with the user input. It is used in the updateProfile() function for updating favGames, password, and phoneNum.

Select * FROM Catalog

```
case 1:
    System.out.print("Enter genre: ");
    String genre = in.readLine();
    query = String.format("SELECT * FROM Catalog WHERE genre = '%s'", genre);
    break;

query = String.format("SELECT * FROM Catalog WHERE price <= %.2f", maxPrice);
break;

sortChoice = Integer.parseInt(sortChoiceStr);
if (sortChoice == 1) {
    query += " ORDER BY price ASC;";
    break;
} else if (sortChoice == 2) {
    query += " ORDER BY price DESC;";
    break;
```

These queries all select all attributes from the Catalog table for a given thing to sort by (genre or maxPrice), and is combined with order by price ascending or descending. This is used in the viewCatalog() function to filter and sort the view of the Catalog for the user.

SELECT price

```
// get the price for this game
String priceQuery = String.format("SELECT price FROM Catalog WHERE gameID='%s';", gameID);
```

The query above selects the price of a game using gameId to get the price of a game. It is used in placeOrder() function.

INSERT INTO RentalOrder

```
String query = String.format(
    "INSERT INTO RentalOrder (login, noOfGames, totalPrice, orderTimestamp, dueDate) " +
    "VALUES ('%s', %d, %.2f, NOW(), NOW() + INTERVAL '7 days') RETURNING rentalOrderID;",
    login, gameIDs.size(), totalPrice
);
```

The query above inserts all rental order information into the RentalOrder table, and is used in the placeOrder() function. A sequence is used to generate the rentalOrderID when there is an insert into the RentalOrder table, so it is able to return the rentalOrderID so that it can be used again to insert into the TrackingInfo and GamesInOrder table.

INSERT INTO GamesInOrder

```
query = String.format(
    "INSERT INTO GamesInOrder (rentalOrderID, gameId, unitsOrdered) VALUES ('%s', '%s', %d);",
    orderID, gameIDs.get(i), unitsOrdered.get(i)
);
```

The query above inserts all games in an order and their information into the GamesInOrder table. It is used in the placeOrder() function.

INSERT INTO TrackingInfo

```
// insert tracking info for the order
query = String.format(
    "INSERT INTO TrackingInfo (rentalOrderID, status, currentLocation, courierName, lastUpdateDate, additionalComments) " +
    "VALUES ('%s', 'Processing', 'Warehouse', 'Courier', NOW(), '') RETURNING trackingID;",
    orderID
);
```

The query above inserts all tracking order information into the TrackingInfo table, and is used in the placeOrder() function. A sequence is used to generate the trackingID when there is an insert into the TrackingInfo table, so a trackingID does not need to be directly inserted here.

SELECT rentalOrderID

```
// get rental orders that match the login
String query = String.format("SELECT rentalOrderID FROM RentalOrder WHERE login = '%s';", login);
List<List<String>> result = esql.executeQueryAndReturnResult(query);
```

This query selects the rentalOrderIDs from the RentalOrder table that matches the login, and allows the function viewAllOrders() to output a user's past rentalOrderIDs.

SELECT rentalOrderID

```
// get orders that match the login, sorted by most recent, limited to 5
String query = String.format("SELECT rentalOrderID FROM RentalOrder WHERE login = '%s' ORDER BY orderTimestamp DESC LIMIT 5;", login);
List<List<String>> result = esql.executeQueryAndReturnResult(query);
```

This query selects the rentalOrderIDs from RentalOrder that matches the login, but orders it by the orderTimestamp and limits it to 5 to get the 5 most recent orders. It is used in the viewRecentOrders() function.

SELECT r.orderTimestamp, r.dueDate, r.totalPrice, t.trackingID

```
// sql query to get order info
String query = String.format("SELECT r.orderTimestamp, r.dueDate, r.totalPrice, t.trackingID " +
    "FROM RentalOrder r " +
    "JOIN TrackingInfo t ON r.rentalOrderID = t.rentalOrderID " +
    "WHERE r.rentalOrderID = '%s' AND r.login = '%s';", rentalOrderID, login);
```

This query selects the orderTimestamp, dueDate, totalPrice, and trackingID from the join of RentalOrder and TrackingInfo to get the order information. It is used in the viewOrderInfo() function.

SELECT c.gameID, c.gameName, g.unitsOrdered

```
// sql query to get games in the order and number of units
query = String.format("SELECT c.gameID, c.gameName, g.unitsOrdered " +
    "FROM GamesInOrder g " +
    "JOIN Catalog c ON g.gameID = c.gameID " +
    "WHERE g.rentalOrderID = '%s';", rentalOrderID);
List<List<String>> gamesResult = esql.executeQueryAndReturnResult(query);
```

This query selects the gameID, gameName, unitsOrdered from the join between Catalog and rentalOrderID to get the information for the games in an order. It is used in the viewOrderInfo() function.

SELECT t.rentalOrderID, t.courierName, t.currentLocation, t.status, t.lastUpdateDate, t.additionalComments

```
// sql query to get tracking information
String query = String.format("SELECT t.rentalOrderID, t.courierName, t.currentLocation, t.status, t.lastUpdateDate, t.additionalComments " +
    "FROM TrackingInfo t " +
    "JOIN RentalOrder r ON t.rentalOrderID = r.rentalOrderID " +
    "WHERE t.trackingID = '%s' AND r.login = '%s';", trackingID, login);
```

This query selects the rentalOrderID, courierName, currentLocation, status, lastUpdateDate, and additionalComments from the join between TrackingInfo and rentalOrderID to get the tracking information that corresponds to a trackingID. It is used in the viewTrackingInfo() function.

SELECT role

```
// get user role
String roleQuery = String.format("SELECT role FROM Users WHERE login = '%s';", login);
```

This query gets the role of a user based on their login, and is used to check if a user is a manager, employee, or customer. It is used in the following functions: `viewProfile()`, `updateTrackingInfo()`, `updateCatalog()`, and `updateUser()`.

SELECT * FROM TrackingInfo

```
// get the order that matches the tracking ID
String checkQuery = String.format("SELECT * FROM TrackingInfo WHERE trackingID = '%s';", trackingID);
int rowCount = esql.executeQuery(checkQuery);
```

This query gets all the information of a `TrackingInfo` row that corresponds to the `trackingID` and is used to check that a row exists in the table for `trackingID`. It is used in the `updateTrackingInfo()` function.

UPDATE TrackingInfo

```
// update the tracking info in the database
String updateQuery = String.format("UPDATE TrackingInfo " +
    "SET status = '%s', currentLocation = '%s', courierName = '%s', " +
    "additionalComments = '%s', lastUpdateDate = NOW() " +
    "WHERE trackingID = '%s';",
    status, currentLocation, courierName, additionalComments, trackingID);
```

This query updates the tracking information based on the manager's input, using `NOW()` for the `lastUpdateDate`. It is used in the `updateTrackingInfo()` function.

SELECT * FROM Catalog

```
// get game info from catalog that matches the gameID
String checkQuery = String.format("SELECT * FROM Catalog WHERE gameID = '%s';", gameID);
int rowCount = esql.executeQuery(checkQuery);
```

This query gets all the information of a `Catalog` row that corresponds to the `gameID` and is used to check that a row exists in the table for `gameID`. It is used in the `updateCatalog()` function.

UPDATE CATALOG

```

case 1:
    System.out.print("Enter new game name: ");
    String gameName = in.readLine();
    updateQuery = String.format("UPDATE Catalog SET gameName = '%s' WHERE gameID = '%s';", gameName, gameID);
    break;
case 2:
    System.out.print("Enter new genre: ");
    String genre = in.readLine();
    updateQuery = String.format("UPDATE Catalog SET genre = '%s' WHERE gameID = '%s';", genre, gameID);
    break;
case 3:
    System.out.print("Enter new price: ");
    String price = in.readLine();
    updateQuery = String.format("UPDATE Catalog SET price = %s WHERE gameID = '%s';", price, gameID);
    break;
case 4:
    System.out.print("Enter new description: ");
    String description = in.readLine();
    updateQuery = String.format("UPDATE Catalog SET description = '%s' WHERE gameID = '%s';", description, gameID);
    break;
case 5:
    System.out.print("Enter new image URL: ");
    String imageURL = in.readLine();
    updateQuery = String.format("UPDATE Catalog SET imageURL = '%s' WHERE gameID = '%s';", imageURL, gameID);
    break;

```

The queries above all update an attribute from Catalog using the gameID using SET. It is used in the updateCatalog() function.

SELECT * FROM Users

```

// get the corresponding info for the user
String userQuery = String.format("SELECT * FROM Users WHERE login = '%s';", userToUpdate);
int rowCount = esql.executeQuery(userQuery);

```

This query gets all the information of a User row that corresponds to the userToUpdate (login of a user a manager wants to update) and is used to check that a row exists in the table for userToUpdate. It is used in the updateUser() and viewProfile() function.

```

case 1:
    System.out.print("Enter new login: ");
    String newLogin = in.readLine();
    updateQuery = String.format("UPDATE Users SET login = '%s' WHERE login = '%s';", newLogin, userToUpdate);
    break;
case 2:
    System.out.print("Enter new password: ");
    String newPassword = in.readLine();
    updateQuery = String.format("UPDATE Users SET password = '%s' WHERE login = '%s';", newPassword, userToUpdate);
    break;
case 3:
    System.out.print("Enter new role: ");
    String newRole = in.readLine();
    updateQuery = String.format("UPDATE Users SET role = '%s' WHERE login = '%s';", newRole, userToUpdate);
    break;
case 4:
    System.out.print("Enter new favorite games: ");
    String newFavGames = in.readLine();
    updateQuery = String.format("UPDATE Users SET favGames = '%s' WHERE login = '%s';", newFavGames, userToUpdate);
    break;
case 5:
    System.out.print("Enter new phone number: ");
    String newPhoneNum = in.readLine();
    updateQuery = String.format("UPDATE Users SET phoneNum = '%s' WHERE login = '%s';", newPhoneNum, userToUpdate);
    break;
case 6:
    System.out.print("Enter new number of overdue games: ");
    int numOverdueGames = Integer.parseInt(in.readLine());
    updateQuery = String.format("UPDATE Users SET numOverDueGames = %d WHERE login = '%s';", numOverdueGames, userToUpdate);

```

The queries above all update an attribute from User using the userToUpdate (login) using SET. It is used in the updateUser() function, which is only for managers.

Extra Credit Screenshots and Explanations

If you did any extra credit, provide screenshots and/or code snippets. Explain how you implemented the extra credit. (triggers/stored procedures, performance tuning, etc)

Triggers

```
-- creat new sequences starting from 4147 to continue from where csv file left off
CREATE SEQUENCE rentalorder_seq START 4147;
CREATE SEQUENCE trackinginfo_seq START 4147;

-- create trigger function for generating rental order IDs
CREATE OR REPLACE FUNCTION generate_rentalorder_id() RETURNS TRIGGER AS $$
BEGIN
    NEW.rentalOrderID := 'gamerentalorder' || nextval('rentalorder_seq');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- create trigger function for generating tracking IDs
CREATE OR REPLACE FUNCTION generate_tracking_id() RETURNS TRIGGER AS $$
BEGIN
    NEW.trackingID := 'trackingid' || nextval('trackinginfo_seq');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- create triggers for rental order ID generation
CREATE TRIGGER rentalorder_id_trigger
BEFORE INSERT ON RentalOrder
FOR EACH ROW
EXECUTE PROCEDURE generate_rentalorder_id();

-- create triggers for tracking ID generation
CREATE TRIGGER tracking_id_trigger
BEFORE INSERT ON TrackingInfo
FOR EACH ROW
EXECUTE PROCEDURE generate_tracking_id();
```

We used triggers to generate the rentalOrderID and trackingID for each order by setting the starting value to 4147 because there are 4146 orders in the rentalorder.csv and we wanted the IDs to continue from that. The function for the trigger concatenates the nextvalue in either the rentalorder_seq or trackinginfo_seq to “gamerentalorder” or “trackingid,” so it is formatted the same way as in the csv files.

Indexes

```
DROP INDEX IF EXISTS idx_users_login;
DROP INDEX IF EXISTS idx_users_role;
DROP INDEX IF EXISTS idx_catalog_price;
DROP INDEX IF EXISTS idx_catalog_gameID;

-- Users table indexes
CREATE INDEX idx_users_login
ON Users
USING BTREE
(login);

CREATE INDEX idx_users_role
ON Users
USING BTREE
(role);

-- Catalog table indexes
CREATE INDEX idx_catalog_gameID
ON Catalog
USING BTREE
(gameID);

CREATE INDEX idx_catalog_price
ON Catalog
USING BTREE
(price);
```

We created more than one index for our queries because login and role from the Users table, as well as gameID and price from the Catalog were used the most. We did not want to add too many indexes for our queries because they could slow down the query performance.

Robust to Unexpected Inputs

```
Please make your choice: 2
What would you like to update?
1. Favorite Games
2. Password
3. Phone Number
4. I am done updating
Enter your choice: 7
Invalid choice!
What would you like to update?
1. Favorite Games
2. Password
3. Phone Number
4. I am done updating

Rental Order ID: gamerentalorder4147
MAIN MENU
-----
1. View Profile
2. Update Profile
3. View Catalog
4. Place Rental Order
5. View Full Rental Order History
6. View Past 5 Rental Orders
7. View Rental Order Information
8. View Tracking Information
9. Update Tracking Information
10. Update Catalog
11. Update User
.....
20. Log out
Please make your choice: 7
Enter rental order ID: abc
Rental order ID abc not found for customer amy
MAIN MENU
```

For our functions, we tried to implement input validation using the switch statement or default case, as well as exception handling with try and catch.

Problems/Findings

A challenge we encountered was creating unique rentalOrderID and trackingID using UUIDs or serial numbers but realized we could just use a trigger to generate the IDs when we insert new rows into rentalorder and trackinginfo tables. Instead of making our own unique numbers, we created a sequence to continue from the values in the csv files for rentalorder.csv and trackinginfo.csv. Another challenge was the role check for functions that only managers or employees can use. Initially, we thought the logic we

had was incorrect for allowing only managers and/or employees to make changes. However, after debugging, we were able to fix the errors by using trim() to remove any whitespace when we retrieved a user's role to check if they are a manager or employee.

Assumptions

Our implementation of the functions assume the following:

- 1) A rental order is due in 7 days/1 week from the orderTimeStamp
- 2) Tracking info for a new rental order has status: Processing, current location: Warehouse, and courier: Courier until it is updated by the manager
- 3) Manager can update all user info through the updateUser() function: login, password, role, favGames, phoneNum, numOverDueGames
- 4) Manager can view the profile of any user with viewProfile()

Contributions

We split up the work evenly, each implementing approximately half of the functions. Amy worked on the following functions: CreateUser(), LogIn(), viewProfile(), updateProfile(), viewCatalog(), placeOrder(), viewAllOrders(). Sarah worked on the following functions: viewRecentOrders(), viewOrderInfo(), viewTrackingInfo(), updateTrackingInfo(), updateCatalog(), updateUser(). Amy also created the file for the triggers.sql, and Sarah created the file for create_indexes.sql.